TagForge

TagForge is a Linux-first command-line pipeline that turns paired-end FASTQ data from antibody–oligonucleotide barcode libraries into corrected Barcode1-by-antibody count matrices. It preserves read-level correction traces, deduplicated molecule details, reproducible saturation analyses, checkpoints, logs, Excel workbooks, and interactive HTML reports.

The implementation is streaming on Python 3.9+. FASTQ and large TSV files are processed incrementally; high-cardinality UMI and matrix aggregation uses temporary SQLite databases. Cutadapt and UMI-tools are mandatory runtime dependencies: linker matching uses Cutadapt's Python API and UMI grouping uses UMI-tools' `UMIClusterer` directly.

TagForge uses patch releases for every code modification (`0.1.1`, `0.1.2`, and so on). `tagforge --version` prints the installed code version, and every pipeline step records `tagforge version=...` in the sample log. Release changes are recorded in `CHANGELOG.md`.

Installation

On Linux, install the mandatory tools in a Conda environment, then install TagForge without letting pip replace the Conda-managed packages:

```
git clone https://github.com/yuanfu-bio/TagForge.git
cd tagforge
conda env create -f environment.yml
conda activate tagforge
python -m pip install -e . --no-deps --no-build-isolation
tagforge validate-config --config configs/config.example.yaml
```

The equivalent manual installation is:

```
conda create -n tagforge -c conda-forge -c bioconda \
  python=3.11 'cutadapt>=4.6' 'umi_tools>=1.1.5' 'pyyaml>=6' \
  'setuptools>=68' wheel
conda activate tagforge
python -m pip install -e . --no-deps --no-build-isolation
```

`validate-config` checks both Python imports and the `cutadapt`/`umi_tools` executables, prints their versions, and fails early with the Conda installation command if either dependency is unavailable.

Five-minute example

```
python examples/generate_example_data.py
tagforge validate-config --config configs/config.example.yaml
tagforge run --config configs/config.example.yaml
```

`tagforge run` performs `quick-test` first by default and starts the formal pipeline only after the small-subset QC succeeds. Disable this explicitly with:

```
tagforge run --config configs/config.example.yaml --skip-quick-test
# --no-quick-test is an equivalent alias
```

The YAML default can also be changed with `quick_test.enabled: false`.

For a fast first look, inspect the leading reads from each FASTQ pair:

```
tagforge quick-test --config configs/config.example.yaml \
  --reads 10000 --threads 4
```

`quick-test` does not create pipeline checkpoints or formal intermediate data. It reports read lengths and N content, per-segment linker/fixed extraction, barcode validity, UMI duplication, top Barcode1/features, speed, and dependency versions. Results are written to `07_report/{sample}.quick_test.tsv`, an HTML summary, and `00_logs/{sample}.quick_test.log`. The default subset size can be set with `quick_test.reads` in YAML and overridden with `--reads`. By default TagForge reads the first 10,000 paired records and stops immediately; it does not scan the rest of a large FASTQ file. Configure the count with `quick_test.reads` or `--reads`. The examined read IDs and 1-based indices are written to `{sample}.quick_test.sampled_read_ids.txt`. The TSV and log report the read count, loading time, and throughput.

The generated input follows the expected convention:

```
01_raw/{sample}/{sample}_raw_1.fq.gz
01_raw/{sample}/{sample}_raw_2.fq.gz
```

Paths themselves are configurable and need not use that layout.

Configuration

Start from `configs/config.example.yaml`. A segment chooses `R1` or `R2`, a target (`barcode1`, `barcode2`, or `umi`), and supports `fixed`, `linker`, or a composition of both. Fixed coordinates in YAML are **0-based half-open intervals**: `start: 6, length: 10` extracts `[6, 16)`. Linker segments accept `left_linker`, `right_linker`, either one alone, and `linker_max_mismatch`. Matching is performed by Cutadapt with full-length overlap and indels disabled, so the setting is an absolute substitution count. When both linkers are present, TagForge enumerates every Cutadapt-verified left and right match (including overlapping occurrences), builds all correctly oriented pairs, and selects the pair with the shortest intervening sequence. Ties use the earliest left match and then the earliest right match. The result is accepted only when that shortest gap is exactly `length`; otherwise the read enters fixed fallback. Multi-hit read counts, candidate-pair totals, and selected gap ranges are written to extraction statistics and logs.

When both methods are configured for one segment, they form a fallback chain: TagForge tries **linker first**; only reads whose linker extraction fails are retried with fixed

extraction. `start` is always a 0-based coordinate on the original R1/R2 sequence. A linker-successful read never enters fixed mode:

```yaml
- name: CELL
  target: barcode1
  read: R1
  methods: [linker, fixed]
  left_linker: AACCT
  right_linker: TGGCA
  linker_max_mismatch: 1
  start: 2       # coordinate on the original read; fallback only
  length: 8
```

The equivalent compact form is to provide `start` and linker fields together; TagForge infers the fallback mode even when `method` contains only one name. `method: linker_fixed`, `method: linker+fixed`, and `methods: [linker, fixed]` are also accepted explicitly. The extracted read-detail table has seven compact columns: `read_id`, `barcode1_segments`, `barcode2_segments`, `umi_segments`, `methods`, `status`, and `failure_reason`. Segment sequences are comma-separated in configuration order; `methods` uses one code per configured segment (`L` = linker, `F` = fixed, `X` = failed). Combined raw-barcode duplicates, JSON syntax, repeated segment names, and repeated method words are not stored. Extracted tables from earlier versions are intentionally rejected: rerun `tagforge extract --overwrite` after upgrading. Checkpoints include the TagForge version so an older checkpoint cannot silently bypass the new extraction step.

Extraction streams to `{sample}.extracted.tsv.gz.tmp` in bounded batches; the `.tmp` suffix means "incomplete", not "held in memory". Each batch is flushed to disk, and only the configured batch plus worker data stays in memory. On success the temporary gzip is atomically renamed. `performance.chunk_size` controls the memory/throughput tradeoff (default: 10,000 read pairs).

During extraction, the first rows are immediately readable from `02_extracted/{sample}.extracted.preview.tsv` (1,000 by default, controlled by `performance.extraction_preview_reads`). Live status is written to `00_logs/{sample}.extraction_progress.tsv` and printed/logged after every batch: completed reads, approximate compressed-input percentage, speed, ETA, estimated finish, and current `.tmp` size. Percentage and ETA are estimates based on physical compressed bytes consumed, so gzip read-ahead and variable compression can

cause small fluctuations. A non-final batch is never displayed as 100%; only confirmed end-of-file reports 100%.

Barcode segments can have independent whitelist and correction controls:

```
correction:
  enabled: true
  allow_shift: true
  max_shift: 1
  allow_mismatch: true
  max_mismatch: 1
```

Correction tries exact, shift, mismatch, and shift-plus-mismatch candidates. Candidates are scored by operation count and total edit distance. A tied best result pointing to multiple whitelist entries is marked ambiguous and rejected. Raw, shifted, and final sequences remain in the trace. Multiple segments of a target are concatenated in configuration order.

For fixed extraction, `max_shift` reserves bases on both sides of the configured interval. With `start: 16`, `length: 8`, and `max_shift: 1`, extraction retains `read[15:25]`. Correction tests the configured interval first (`raw[1:9]`), then the left (`raw[0:8]`, shift `-1`) and right (`raw[2:10]`, shift `+1`) candidates. The signed distance is preserved in the correction trace; aggregate statistics also report left and right shift counts separately. Linker-derived barcodes are already delimited and therefore do not receive positional shifting.

`FB_info.tsv` must contain the configured ID, sequence, and antibody-name columns. FB sequences must be unique. Antibody names must also be unique unless `allow_duplicate_names: true` is explicitly configured.

Commands

```
# One sample or every configured sample
tagforge run --config 00_config/config.yaml --sample sample-A --threads 8
tagforge run --config 00_config/config.yaml --threads 8

# Individual stages
tagforge extract    --config 00_config/config.yaml --sample sample-A
tagforge correct    --config 00_config/config.yaml --sample sample-A
tagforge dedup      --config 00_config/config.yaml --sample sample-A
tagforge matrix     --config 00_config/config.yaml --sample sample-A
```

```
tagforge downsample --config 00_config/config.yaml --sample sample-A
tagforge report     --config 00_config/config.yaml --sample sample-A

tagforge init-config --out 00_config/config.yaml
tagforge make-slurm --config 00_config/config.yaml --out slurm_jobs \
  --partition compute --account my_lab --qos normal \
  --threads 8 --mem 16G --time 24:00:00 --conda-env tagforge
```

`--sample` is repeatable. `--overwrite` ignores successful checkpoints. `make-slurm` also accepts `--constraint`, `--gres`, `--nodes`, `--ntasks`, `--mail-user`, `--mail-type`, and repeatable `--extra-sbatch` options. Generated jobs activate the requested Conda environment before running TagForge.

Outputs

Each sample gets these directories under `02_output/{sample}`:

```
00_logs/       pipeline log
01_checkpoint/ successful-step markers
02_extracted/  compressed extraction detail
03_corrected/  correction trace and statistics
04_matrix/     raw and optimal-saturation matrices
05_detail/     valid reads and molecule details
06_downsample/ saturation metrics and optimum
07_report/     Excel and HTML reports
08_tmp/        disk-backed aggregation scratch space
```

The matrix rows are final Barcode1 values and columns are antibody names from the annotation. Counts are deduplicated corrected UMIs. `00_report/` at the project root contains the batch workbook (`meta` and bulk `counts` sheets) and batch HTML overview.

`02_extracted/{sample}.extraction_stats.tsv` contains per-segment Cutadapt QC: linker attempts, successes and failures, success rate, failure reasons, fixed fallback attempts/rescues, final success rate, workers, parallel backend, wall-clock time, cumulative Cutadapt matching CPU time, and throughput. The same information is emitted to `00_logs/{sample}.pipeline.log` and included in the sample Excel/HTML reports.

The HTML report loads Plotly from its pinned CDN URL. The Excel writer is dependency-free and produces normal `.xlsx` workbooks with styled headers, filters, and frozen header rows.

UMI and saturation definitions

UMIs are grouped by UMI-tools within each final Barcode1–antibody pair. Directional mode uses an edge from a more abundant UMI `A` to a neighbor `B` when their Hamming distance is within the threshold and `count(A) >= 2*count(B)-1`.

Downsampling independently retains every supporting read with probability `ratio`, using a SHA-256-derived seed from the configured seed, sample, ratio, and repeat. It is reproducible across Python processes.

With `downsample.ratios: auto` (also the default when `ratios` is omitted), the pipeline evaluates 36 ratios generated by multiplying each base in `[0.0001, 0.001, 0.01, 0.1]` by integers 1 through 9. This covers `0.0001–0.0009`, `0.001–0.009`, `0.01–0.09`, and `0.1–0.9`. An explicit YAML list remains supported when a custom grid is needed.

- `UMI Types`: corrected molecules with at least one sampled read.
- `UMI detected once`: those molecules supported by exactly one sampled read.
- `duplication_ratio`: `(sampled reads − UMI types) / sampled reads × 100`.
- `sequencing_saturation`: `(1 − singleton UMIs / UMI types) × 100`.

The ratio with maximum saturation is selected; ties prefer the smaller ratio, then the smaller repeat. Its molecule detail and count matrix are regenerated with the identical deterministic seed.

Resume, logs, and failure behavior

A versioned checkpoint is atomically written only after all expected stage outputs exist and are non-empty. A later run skips that stage only when the checkpoint version matches and all outputs remain present. Important outputs are first written with a `.tmp` suffix and atomically renamed.

Failures include sample and stage context in `00_logs/{sample}.pipeline.log`. Configuration validation catches missing inputs, malformed segments, unsupported UMI methods, duplicate samples, invalid ratios, missing annotation columns, and whitelist problems. FASTQ parsing checks four-line structure, sequence/quality lengths, mate counts, and matching read IDs.

Performance tuning

- Put `08_tmp` on fast local storage when processing large libraries.
- Increase `performance.chunk_size` to reduce extraction scheduling overhead when memory permits; lower it to tighten the extraction memory bound and receive more frequent progress updates.
- Lower `compression_level` for faster output on compute-heavy runs.
- Disable the large correction trace with `output.correction_trace: false`.
- Split samples into separate Slurm jobs with `make-slurm`.

`performance.threads` and the overriding CLI option `--threads` control the number of process workers used for Cutadapt-backed linker extraction. Process workers are used instead of Python threads so all allocated Slurm CPUs can be used reliably; output order remains identical to FASTQ input order. The chosen worker count and actual backend are recorded in the sample pipeline log. On a restricted runtime that blocks process/semaphore creation, TagForge falls back to a Cutadapt thread pool; normal Linux and Slurm jobs use process workers. Because matching uses Cutadapt's Python API, Linux process listings show TagForge Python workers rather than a separate `cutadapt --cores` command; each worker is nevertheless executing Cutadapt's matching engine.

Read-level files are never loaded wholesale. UMI grouping holds one Barcode1–feature group at a time. Report creation only loads summary tables; feature totals and molecule counts are streamed.

Troubleshooting and FAQ

A linker is not found. Check orientation, linker sequence, segment length, and `linker_max_mismatch`. TagForge delegates matching to Cutadapt with full linker overlap and substitutions only.

Many corrections are ambiguous. The whitelist entries may be too close for the chosen mismatch threshold. Lower `max_mismatch` or redesign the whitelist.

Can Barcode1, Barcode2, or UMI span both reads? Yes. Define multiple segments in their desired concatenation order; each segment independently selects R1 or R2.

Why can saturation peak before 100%? The requested definition measures the fraction of observed molecules that are non-singletons. Downsampling changes both its numerator and denominator, so TagForge evaluates every configured ratio rather than assuming the full library is optimal.

How do I start over? Prefer `--overwrite`. Removing selected outputs and their matching checkpoint also causes only that stage to rerun.

Tests

```
python -m unittest discover -s tests -v
# or, with the dev extra
pytest
```

The suite covers extraction, exact/mismatch/shift/combined correction, ambiguity, paired FASTQ parsing, directional UMI grouping, metrics, config validation, CLI behavior, and the example end-to-end pipeline. Without the Conda environment, external-tool calls are mocked in unit tests and the real end-to-end test is skipped; inside the environment, all tests run against Cutadapt and UMI-tools.